Personal Indian Stock Tracker — Development Plan

*A complete guide to building the app. Written so that even someone with zero stock-market knowledge can understand what we're building and how to build it. This is a **web-only** application.*

Part 0 — What are we even building? (Read this first)

We are building a **private website that shows live stock prices for the Indian market**. Only the owner uses it — it's not for the public, and it runs on the **web only** (not a mobile app).

If you don't know the stock market, here's all you need:

- A **stock** (or "share") is a tiny piece of ownership in a company. If you own a share of Reliance, you own a microscopic slice of Reliance.
- Every publicly listed company has a short code called a **ticker symbol** — for example `RELIANCE`,

`TCS`, `INFY`, `HDFCBANK`.

- During market hours, the **price** of each stock changes constantly — up and down, second by second, in rupees (₹).
- Think of the whole thing like a **live scoreboard for companies**. Each row is a company, and the number next to it is its current price, turning green when it goes up and red when it goes down.

What our app does:

1. Shows a **watchlist** — a list of stocks the owner cares about, with live prices.
2. Shows a **detail page** for any stock — a candlestick price chart, current price, day's high/low, and technical indicators.
3. Shows a **portfolio** — stocks the owner already holds, and whether they're in profit or loss.

Where do the live numbers come from?
From a stockbroker called **Angel One**, through their free developer service called **SmartAPI**. The owner already has an Angel One account. SmartAPI is free, including live data.

When is the market open?
Monday to Friday, roughly **9:15 AM to 3:30 PM Indian time**. Closed on weekends and on ~14–16 public holidays a year. Outside these hours, prices simply don't change — that's normal, not a bug.

Part 1 — The big picture (how the pieces fit together)

Imagine a **restaurant**. That's the easiest way to understand the parts:

Part	Restaurant analogy	In our app
Frontend	The dining room the customer sees	The website screens — buttons, charts, lists
Backend	The kitchen that does the work	A program that fetches data and does the thinking
SmartAPI	The supplier delivering fresh ingredients	Angel One sending live stock prices
Database	The pantry / storage room	Where we keep data (watchlists, past prices, built candles)

The data flows in one direction:

```
Angel One (SmartAPI)  →  Our Backend  →
Our Frontend  →  The Owner's screen
   (live prices)         (the brain)
(the display)
```

So the **backend** talks to Angel One, and the **frontend** talks to the backend. The frontend never talks to Angel One directly.

Part 2 — The technology we'll use (and why)

Everything here is **free**.

Frontend (what the owner sees)

- **Next.js** — a popular framework built on React. Gives us pages and structure without building everything from scratch.
- **TypeScript** — JavaScript with type-checking. Prevents silly bugs, which matters a lot with financial numbers.
- **Tailwind CSS** — lets us style the app quickly without writing tons of CSS.
- **TradingView Lightweight Charts** — a free, ready-made charting tool for the candlestick chart. *We do not build charts from scratch.*

Backend (the brain)

- **Python** — the language we'll write the backend in. Chosen because Angel One's best, most mature SDK is for Python.
- **FastAPI** — a modern Python web framework. It's fast and handles live streaming well.
- **smartapi-python** — Angel One's official Python library for connecting to SmartAPI.
- **pyotp** — a small library to generate the login security code automatically (explained in Part 4).
- **pandas-ta** (or **TA-Lib**) — a Python library that calculates technical indicators like moving averages and TRIX (explained in Part 3.5).

Storage

- **PostgreSQL** — a database for things we keep long-term (watchlists, portfolio, past price history, self-built candles).
- **Redis** — a super-fast temporary store, used for live prices and for pushing updates to the screen instantly.

Where it runs (hosting) — also free

- **Simplest:** run it on the owner's own computer. Costs ₹0. Only runs while the computer is on.

- **Always-on:** Oracle Cloud's "Always Free" tier — a free server that runs 24/7 forever, far more powerful than this app needs.

Part 3 — What the app should actually DO (the features / screens)

Build these screens. Start simple, add polish later.

Screen 1 — Watchlist (the home screen)

- A list of stocks the owner is watching.
- Each row: ticker symbol, company name, current price, and the day's change (e.g. `+1.2%` in green or `-0.8%` in red).
- Prices update live while the market is open.
- An "add stock" and "remove stock" button.

Screen 2 — Stock Detail

- Opens when the owner taps a stock.
- A big **candlestick price chart** from TradingView, with **green and red candles**.
- **Timeframe buttons:** `15s` · `1m` · `5m` · `15m` — switches how much time each candle covers (full detail in Part 3.5).

- **Indicator toggles: Moving Average (EMA) and TRIX** — drawn on top of the chart (see Part 3.5).
- Current price, day's high, day's low, opening price.
- A **volume** bar chart underneath the price chart.

Screen 3 — Portfolio

- A list of stocks the owner actually owns.
- For each: quantity held, buy price, current price, and **profit/loss** (in ₹ and %).
- A total at the bottom: overall profit or loss.

Screen 4 — Alerts (optional, add later)

- Owner sets a target, e.g. "tell me when TCS goes above ₹4000."
- The app notifies them when it happens.

Part 3.5 — Candlestick charts: timeframes, colors & indicators

This is the heart of the stock screen, so it gets its own section.

What a candlestick is (for the newcomer)

Each candle represents price movement over **one fixed slice of time**. It shows four numbers, called

OHLC:

- **Open** — the price at the start of the slice
- **High** — the highest price during the slice
- **Low** — the lowest price during the slice
- **Close** — the price at the end of the slice

Visually:

- The thick part (the **body**) spans from open to close.
- The thin lines above and below (the **wicks**) reach up to the high and down to the low.
- **Green candle** = price closed **higher** than it opened (went up in that slice).
- **Red candle** = price closed **lower** than it opened (went down in that slice).

Good news: the TradingView chart colors candles green/red **automatically**. You just feed it the OHLC numbers with timestamps — you don't color each candle by hand. (You can customize the exact shades if you want.)

The timeframes: 15s, 1m, 5m, 15m

A "timeframe" is how much time each candle covers:

- **1 minute** = one new candle forms every minute.

- **5 minutes** = one candle every 5 minutes, and so on.
- Smaller timeframe = more candles, more detail, more zoomed-in. Larger = fewer candles, smoother, more zoomed-out.

The owner switches between them with the timeframe buttons.

⚠️ Important: the truth about the 15-second candle

It is **not** the case that "the website has 15s and mobile has 1m." Both would pull from the same source (Angel One SmartAPI), and here's what that source actually gives:

> Angel One's ready-made candle data only goes down to **1 minute**. The available intervals are: 1 min, 3 min, 5 min, 10 min, 15 min, 30 min, 1 hour, 1 day. **There is no ready-made 15-second candle.**

So:

- **1m, 5m, 15m** → request them directly from Angel One's historical data API. Easy. ✅
- **15 seconds** → you must **build it yourself**. ⚠️

Why our web app can still do 15s (and a basic app can't): building 15-second candles needs a program that continuously listens to the live price feed and

assembles candles in real time. That's exactly what our web app's **backend** does. A simple mobile app usually just displays whatever the broker hands it (1-minute and up), so it "only has 1m." Our web app goes finer **because we build it ourselves** — not because the website format magically supports it.

How to build a 15-second candle:
Take the live price feed (the WebSocket that streams the last traded price) and group incoming prices into 15-second buckets. For each bucket:

- **Open** = the first price that arrived in the bucket
- **High** = the highest price in the bucket
- **Low** = the lowest price in the bucket
- **Close** = the last price in the bucket

Then color it green if close ≥ open, red if close < open — same rule as any candle.

Two catches with self-built 15s candles:

1. **Forward-only.** You can only build them from the moment you start listening. You cannot fetch *past* 15-second candles from Angel One. If the owner wants to keep them, save them to the database as they form.
2. **Depends on activity.** For heavily-traded stocks, prices arrive many times a second, so 15s

candles look full. For thinly-traded stocks, few prices arrive, so candles may be sparse.

The "live forming" candle (easy to miss)

On the short timeframes especially, the **most recent candle is still forming** — it keeps changing as new prices arrive, right up until its time slice ends. So the chart should **update the last candle live** on each new price, and only add a brand-new candle when the slice completes. (TradingView Lightweight Charts has an `update()` call for exactly this.) Without it, the newest candle looks frozen and wrong.

Volume

Each candle also comes with a **volume** number (how many shares traded in that slice). It's standard to show a small bar chart of volume underneath the price chart. Angel One's candle data already includes volume, so it's easy to add.

Indicators: Moving Average and TRIX

Indicators are lines drawn on top of the chart to help read the price. We're adding two:

Moving Average (MA) — the simplest and most popular indicator. It's just the **average price over the last N candles**, drawn as a smooth line. Two flavors:

- **SMA** (Simple) — a plain average.

- **EMA** (Exponential) — weights recent prices more, so it reacts faster.

It's used to see **trend direction** (price above the line = generally rising) and **crossovers** (a short MA crossing a long MA is a common signal). Easy to read, even for a beginner.

TRIX — short for **Triple Exponential Average**. It's a **momentum** indicator: it smooths the price three times to strip out noise, then shows how fast that smoothed price is changing, as a line that wiggles around a zero level. Crossing zero (or its signal line) is read as a momentum shift. It's better at ignoring small random moves, but it **lags** and is harder to interpret.

Which is better for tracking a stock?

- If you're new to this, **start with a moving average (an EMA)** — it's intuitive, widely used, and enough to see the trend at a glance.
- **Add TRIX later** as a second layer, once comfortable — good for filtering out noise and spotting momentum turns.
- **Honest, important point:** *no indicator predicts prices reliably.* Indicators summarize **past** price behavior; they don't tell the future. Traders use them as one input among many and combine several rather than trusting one. And on very short timeframes like **15-second candles, indicators**

get extremely noisy and throw lots of false signals — they were designed for longer timeframes. Treat them as hints, not answers. *(This describes how the tools behave; it is not financial or investment advice.)*

How to add them technically:
TradingView Lightweight Charts draws candles and lines but does **not** calculate indicators for you. So either:

- Compute the values yourself with a library — `pandas-ta` or `TA-Lib` in Python (backend), or `technicalindicators` in JavaScript (frontend) — then plot each indicator as an extra line series, **or**
- Use TradingView's fuller **Advanced Charts** library, which has moving averages, TRIX, and dozens of indicators built in. (It's free but requires a quick application to access.)

Part 4 — How the data flows (the most important technical part)

This is the part that first-time builders get wrong, so read carefully.

The login has a quirk

To connect to SmartAPI, the backend logs in using three things:

1. The owner's **client code**
2. Their **PIN**
3. A **TOTP** — the 6-digit code that changes every 30 seconds (like the codes in Google Authenticator).

Two things to handle:

- The TOTP must be **generated automatically in code** using the `pyotp` library — the owner shouldn't have to type it in.
- The login **session expires at midnight** every night. So the backend must **log in again automatically each day**.

The golden rule: ONE connection, MANY screens

There is only **one** Angel One account, but the owner might have several screens or tabs open. **Do not open a separate Angel One connection for each screen.** Instead:

```
Angel One  →  Backend keeps ONE live
connection  →  Backend puts prices into
Redis

↓
                        Redis instantly
```

```
pushes those prices out to every open
screen
```

Back to the restaurant analogy: there's **one delivery of ingredients** (one Angel One feed), and the kitchen distributes it to **all the tables** (all the screens).

The tool that pushes live prices to the screen is called a **WebSocket** — think of it as a phone line that stays open, so the backend can "call up" the screen the moment a price changes, instead of the screen repeatedly asking "any update? any update?"

Note: this same live feed is what we use to build the 15-second candles (Part 3.5) and to update the live forming candle.

Part 5 — The build order (do it in THIS sequence)

Don't try to build everything at once. Follow these phases. Each one should fully work before moving to the next.

Phase 1 — Setup

- Install the tools (Python, Node.js, PostgreSQL, Redis).

- Create a SmartAPI app on the Angel One developer portal to get the API key.
- Get a basic empty Next.js page and empty FastAPI backend both running ("Hello World").

Phase 2 — Backend login

- Make the backend log in to SmartAPI (client code + PIN + auto-generated TOTP).
- Fetch the price of ONE stock (e.g. Reliance) and print it. 🎉 First real milestone.

Phase 3 — Backend data

- Fetch **historical** candles (1m / 5m / 15m) for charts, and **live** prices.
- Store historical data in PostgreSQL; cache live prices in Redis.
- Set up automatic daily re-login.

Phase 4 — Frontend skeleton

- Build the three screens as static layouts (fake numbers for now).
- Set up navigation between them.

Phase 5 — Connect frontend to backend

- Make the frontend call the backend and show **real** prices instead of fake ones.
- The watchlist and portfolio now show live data.

Phase 6 — Real-time + charts + 15-second candles

- Add the WebSocket so prices update **instantly** without refreshing.
- Plug in the TradingView candlestick chart with the `15s / 1m / 5m / 15m` timeframe buttons.
- Build the **15-second candle** aggregator from the live feed, and the **live forming candle** update.
- Add the **volume** bars.

Phase 7 — Indicators, portfolio & polish

- Add the **moving average (EMA)** and **TRIX** indicators to the chart.
- Add the portfolio profit/loss calculations.
- Add alerts (optional).
- Improve the design, colors, and layout.

Part 6 — Things unique to the Indian market (don't get caught out)

These are traps that generic tutorials won't warn you about:

- **Two exchanges:** India has two — NSE and BSE. The same company can be on both, sometimes with slightly different symbols. Pick one (NSE is the most common) to keep it simple at first.

- **Holidays:** The market is closed on ~14–16 holidays a year *plus* weekends. Build a small "trading calendar" so the app doesn't expect data on Diwali or Republic Day.
- **Circuit limits:** If a stock moves too fast, the exchange freezes it. The price stops updating and the feed goes quiet — this is **normal**, not a crash. Handle it gracefully.
- **Corporate actions:** When a company does a stock split or bonus, old prices need adjusting or the chart looks broken. Use "adjusted" prices where the data provider offers them.
- **Timezone:** Indian market time is IST (UTC + 5:30). Don't copy assumptions from US-market tutorials.
- **A ~5-second delay on finished candles:** When pulling just-completed candles from the historical API, the latest one takes about 5 seconds to appear. That's why we combine two sources — load past candles from the historical API, and update the live/forming candle from the WebSocket.
- **The April 2026 rule:** If you ever add the ability to *place real buy/sell orders*, Angel One now requires a registered **static IP**. But for our app — which only *displays* data — this rule does **not** apply. Don't add order-placing unless you're ready to deal with it.

Part 7 — Splitting the work (if two people build it)

The frontend and backend meet at a clear boundary called the **API** — a set of web addresses (URLs) where the backend promises to return certain data. Agree on these first, then each person can work independently.

Example agreement (the "contract"):

- `GET /watchlist` → backend returns the list of watched stocks with current prices
- `GET /stock/RELIANCE?interval=5m` → backend returns candle data for one stock at a timeframe
- `GET /portfolio` → backend returns holdings with profit/loss
- A **WebSocket** channel → backend streams live prices and forming candles

Once the contract is agreed:

- **Frontend person** builds the screens against these URLs (using fake sample data until the backend is ready). Good fit if you enjoy design and layout.
- **Backend person** builds the SmartAPI connection, candle-building, indicators, and makes those URLs return real data. Good fit if you enjoy logic and data.

This way, neither person is blocked waiting for the other.

Part 8 — How long will it take?

Rough estimates for the **web-only** app (they depend heavily on experience):

Who's building	Realistic time
Comfortable with React + Python	**~4–6 weeks** of focused work
Learning as you go	**~2–3 months**

Rough split of effort:

- Setup + SmartAPI login → ~1 week
- Data + storage → ~1 week
- The three screens → ~1 week
- Real-time + charts + 15-second candles → ~1–2 weeks
- Indicators + portfolio + polish → ~1–2 weeks

The first big win comes early — getting one live price to print in **Phase 2**. After that, progress feels much faster.

Part 9 — Before you start: the checklist that saves you pain

These aren't features — they're the operational things that cause the most grief if skipped. Sort them out early.

1. **Protect your login details (most important).** The API key, client code, PIN, and especially the **TOTP secret** must never be hardcoded in the code or pushed to GitHub. Anyone who gets the TOTP secret could log into the actual trading account. Keep them all in a separate secrets / `.env` file that is **git-ignored from day one**.
2. **Get the "scrip master" (symbol tokens).** SmartAPI doesn't identify stocks by name — it uses **numeric tokens** (Reliance = a specific number). Early on, download Angel One's instrument list (the **ScripMaster JSON**) and build a lookup that maps `RELIANCE` → its token. It's easy to overlook, and nothing works without it.
3. **You can only fully test while the market is open.** Live prices, the WebSocket, forming candles, and the self-built 15-second candles only work **9:15 AM–3:30 PM IST on weekdays**. Off-hours you'll just see static data — so plan live-testing during market hours, or record some live data to replay later.

4. **Handle reconnects and daily re-login.** WebSocket connections drop, and the login session dies at midnight. The app must **auto-reconnect and re-login on its own**, or it'll silently stop updating. Also respect SmartAPI's **rate limits** — don't refresh too aggressively; cache instead.
5. **Back up the database.** The 15-second candles **can't be re-fetched** once missed, and the free Oracle tier has **no uptime guarantee** — so a simple **daily backup** saves you from losing built-up data.

Quick summary

We're building a **private, web-only live stock-tracker** for the Indian market. The **backend** (Python + FastAPI) connects to **Angel One's free SmartAPI**, keeps one live feed, stores/caches data, and **builds 15-second candles itself** (Angel One only provides 1-minute and up). The **frontend** (Next.js) shows a watchlist, a candlestick chart with `15s / 1m / 5m / 15m` timeframes, **green/red candles**, **volume**, and **moving-average + TRIX** indicators, plus a portfolio — all updated live over a WebSocket. It's **free to build and run**, takes roughly **4–6 weeks** for an experienced developer, and should be built **phase by phase**, starting with "fetch and print one stock price." Before

coding, lock down the **Part 9 checklist** (protect login details, get symbol tokens, test during market hours, handle reconnects, back up data). Remember: indicators are **hints, not predictions**, and a few **India-specific quirks** (two exchanges, holidays, circuit limits) need handling along the way.